Multiplicative Cognition: Prime Memory Algebra (PMA)

Executive summary

PMA treats **addresses** as (conceptually) primes and **stored values** as exponents on those addresses. The system is implemented as a sparse exponent map (not as a huge integer), while the Gödel-style multiplicative code is used as a *view* and/or an *integrity commitment target*.

This document formalizes:

- the **state space** and **operators** (read/write/merge/meet/join),
- a **kernel** for symbolic retrieval and compositional querying,
- a **CRDT replication** discipline for multi-agent shared memory,
- an **audit/commitment interface** for verifiable agent state.

---

1) Mathematical formalization

1.1 Address space

Let A be a countable address set. Conceptually $A = \mathbb{P}$ (the primes), but in implementations A is a set of 64-bit identifiers derived from a deterministic registry or hash-to-ID scheme.

1.2 Working state (two-channel)

To support inhibition without breaking lattice semantics, represent state as a **signed activation** via two nonnegative channels:

$$ e = e^+ - e^- ,\qquad e^+, e^- \in \mathbb{N}^{(A)} $$

where $\mathbb{N}^{(A)}$ denotes finite-support functions $A \to \mathbb{N}$ (sparse maps).

- **Read:** $\mathrm{READ}(a) = e^+(a) - e^-(a)$
- **Positive write:** $e^+(a) \leftarrow e^+(a) + \Delta$
- **Negative write:** $e^-(a) \leftarrow e^-(a) + \Delta$

Squarefree symbolic mode is the restriction $e^+(a) \in \{0, 1\}$, $e^- = 0$.

1.3 Order and lattice operators (positive channel)

Define partial order on $\mathbb{N}^{(A)}$:

$$ x\preceq y \iff \forall a\in A,\ x(a)\le y(a). $$

Then $(\mathbb{N}^{(A)}, \preceq)$ is a distributive lattice with:

$$ (x\wedge y)(a)=\min(x(a),y(a)),\qquad (x\vee y)(a)=\max(x(a),y(a)). $$

Interpretations (positive channel):

- **meet** $\wedge$: shared structure / overlap
- **join** $\vee$: union / combined constraints

1.4 Additive reinforcement monoid

Define reinforcement merge:

$$ (x\oplus y)(a)=x(a)+y(a) $$

so $(\mathbb{N}^{(A)}, \oplus)$ is a commutative monoid.

Important: $\oplus$ is not idempotent, so it is *not* safe as a distributed merge operator unless mediated by a CRDT design (Section 3).

1.5 Gödel-style multiplicative view (conceptual)

If $A = \mathbb{P}$, define

$$ \mathrm{Code}(x)=\prod_{p\in\mathbb{P}} p^{x(p)} \in \mathbb{N}. $$

Then unique factorization yields:

$$ \gcd(\mathrm{Code}(x),\mathrm{Code}(y))=\mathrm{Code}(x\wedge y),\quad \mathrm{lcm}(\mathrm{Code}(x),\mathrm{Code}(y))=\mathrm{Code}(x\vee y). $$

In practice, store x and compute $\wedge$, $\vee$ directly.

---

2) Retrieval kernel and scoring

2.1 Weighted overlap (histogram intersection kernel)

For weights $w : A \to \mathbb{R}_{\geq 0}$, define similarity:

$$ k(x,y)=\sum_{a\in A} w(a)\min(x(a),y(a)). $$

This is the weighted histogram intersection kernel. It favors compositional conjunctions: adding constraints preserves overlap structure via $\min$.

2.2 Entailment / containment (symbolic mode)

In squarefree mode, represent a conjunction of features as $S \subseteq A$. Then:

- X entails Y iff $S_Y \subseteq S_X$.
- shared evidence is $S_X \cap S_Y$.

---

3) Multi-agent shared memory: CRDT replication

3.1 Why naive additive merge fails

If two replicas both apply increments and then simply add received states, retries/duplication double-count. You need idempotent merge.

3.2 G-counter per address (convergent increments)

Let replicas be R. For each address a, store a vector of per-replica counters:

$$ c_a: R\to\mathbb{N}. $$

- Local increment at replica r: $c_a(r) \leftarrow c_a(r) + \Delta$
- Merge: componentwise max

$$ (c_a\sqcup d_a)(r)=\max(c_a(r),d_a(r)). $$

- Readout exponent: $x(a) = \sum_{r \in R} c_a(r)$.

This yields eventual consistency under arbitrary message order, duplication, and partial delivery.

3.3 PN-counter (optional decrements)

To support decrement/decay via CRDT, use two G-counters per address (P and N) and read $x = P - N$. Many systems implement decay as periodic local normalization rather than distributed decrements.

---

4) Auditable agent state: commitments and proofs

4.1 Commitment target

Commit to the sparse map x (or e^+, e^-) with an authenticated data structure. Two pragmatic routes:

(A) Sparse Merkle Tree (SMT)

- leaf key = address id (prime ID)
- leaf value = exponent

- commitment = Merkle root
- slot proof = Merkle path

(B) Vector commitment / SNARK wrapper

- prove correct updates and/or compress proofs

4.2 Verified updates

An update is a transition $x_t \to x_{t+1}$ with an accompanying proof that:

- only specified addresses changed,
- changes match allowed rules (increment, capped increment, decay step, etc.).

4.3 Audit trail

Maintain a chain of roots $\{\rho_t\}$ (optionally hashed together). A decision at time t can be audited by presenting ρ_t plus selective slot proofs.

---

5) Regimes: architecture + validation suites

(i) Symbolic retrieval

Architecture bias

- Use squarefree or low-cardinality exponents.
- Primary ops: $\wedge$, $\vee$, entailment checks, overlap kernel.
- Index: inverted index keyed by address id; store exponent maps sparsely.

Validation suite

1. Compositional query retrieval: conjunction queries with distractors.
2. Constraint propagation: entailment filtering and $\wedge$-based matching.
3. Robustness to ontology growth: add new features over time; verify stable retrieval.

Success criteria

- Higher MRR/top-k on compositional queries than dot-product baselines for sparse features.
- Predictable entailment behavior with low false positives.

(ii) Multi-agent shared memory

Architecture bias

- CRDT counters per address; shard address space across agents.
- Prefer monotone increments; handle decay locally via periodic normalization.
- Merge semantics are deterministic and idempotent.

Validation suite

1. Convergence under unreliable messaging (loss/duplication/reordering).
2. Conflict rate for disjoint shards (should be effectively zero).
3. Throughput vs. merge overhead: sparse deltas and batching.

Success criteria

- Exact convergence to the same logical state across replicas.
- No double-counting under duplicated deliveries.

(iii) Auditable agent state

Architecture bias

- Sparse Merkle commitment for state snapshots.
- Logged update deltas with proofs (slot proofs + optional transition proofs).
- Policy constraints enforced at proof layer (e.g., bounded increments).

Validation suite

1. Selective disclosure: prove $x(a) = k$ without revealing other slots.
2. Tamper detection: adversarial edits must fail verification.
3. Long-run update viability: many updates without proof-size blow-up.

Success criteria

- Fast verification; small proofs.
- Reliable audit reconstruction for decisions.

---

6) Reference code snippets

6.1 Minimal PMA working state (two-channel)

```
from collections import defaultdict
from math import floor

class PMAState:
    """Sparse two-channel exponent memory."""
    def __init__(self):
        self.pos = defaultdict(int)  # e^+
        self.neg = defaultdict(int)  # e^-

    def read(self, a):
        return self.pos.get(a, 0) - self.neg.get(a, 0)
```

```
    def inc(self, a, delta=1):
        self.pos[a] += int(delta)

    def inhibit(self, a, delta=1):
        self.neg[a] += int(delta)

    def meet_pos(self, other):
        out = PMAState()
        keys = set(self.pos) | set(other.pos)
        for a in keys:
            out.pos[a] = min(self.pos.get(a, 0), other.pos.get(a, 0))
            if out.pos[a] == 0:
                out.pos.pop(a, None)
        return out

    def join_pos(self, other):
        out = PMAState()
        keys = set(self.pos) | set(other.pos)
        for a in keys:
            out.pos[a] = max(self.pos.get(a, 0), other.pos.get(a, 0))
            if out.pos[a] == 0:
                out.pos.pop(a, None)
        return out

    def score_overlap(self, query, w=None):
        """Weighted histogram intersection on positive channel."""
        w = w or {}
        s = 0.0
        for a, qv in query.pos.items():
            mv = self.pos.get(a, 0)
            if mv:
                s += float(w.get(a, 1.0)) * min(qv, mv)
        return s

    def decay(self, lam=0.98):
        """Local forgetting/decay."""
        for d in (self.pos, self.neg):
            for a in list(d.keys()):
                d[a] = floor(lam * d[a])
                if d[a] <= 0:
                    del d[a]
```

6.2 CRDT G-counter per address (sketch)

```
from collections import defaultdict
```

```
class GCounter:
    def __init__(self, replica_id):
        self.rid = replica_id
        self.counts = defaultdict(int)  # counts[addr][rid] stored externally;
simplified here

class AddrGCounter:
    """Per-address G-counter vector; merge by per-replica max."""
    def __init__(self):
        self.v = defaultdict(lambda: defaultdict(int))  # v[addr][rid] = int

    def inc(self, addr, rid, delta=1):
        self.v[addr][rid] += int(delta)

    def merge(self, other):
        for addr, vec in other.v.items():
            my = self.v[addr]
            for rid, val in vec.items():
                if val > my.get(rid, 0):
                    my[rid] = val

    def read(self, addr):
        return sum(self.v[addr].values())

    def sparse_delta(self, since_snapshot):
        """Optional: compute deltas for efficient networking."""
        delta = AddrGCounter()
        for addr, vec in self.v.items():
            for rid, val in vec.items():
                if val > since_snapshot.v[addr].get(rid, 0):
                    delta.v[addr][rid] = val
        return delta
```

6.3 Toy benchmark harness for symbolic retrieval

```
def topk_retrieve(query, items, k=10, weights=None):
    scored = [(i, item.score_overlap(query, weights)) for i, item in
enumerate(items)]
    scored.sort(key=lambda x: x[1], reverse=True)
    return scored[:k]

# Example usage:
# items = [PMAState() ...]
# query = PMAState(); query.inc(addrA); query.inc(addrB)
# print(topk_retrieve(query, items, k=5))
```

6.4 Sparse Merkle commitment (interface only)

```python
class SparseMerkleCommitment:
    """Interface sketch: implement with a real SMT library in production."""
    def __init__(self):
        self.root = None

    def commit(self, kv_pairs):
        """kv_pairs: {addr_id: exponent}. Returns root hash."""
        raise NotImplementedError

    def prove_slot(self, addr_id):
        """Returns (value, proof_path) for addr_id."""
        raise NotImplementedError

    @staticmethod
    def verify_slot(root, addr_id, value, proof_path):
        """Verifies membership/non-membership proof."""
        raise NotImplementedError
```

---

7) Implementation notes (non-negotiables)

- Store sparse maps, not $\prod p^{e(p)}$.
- If you need distributed merge: use CRDT semantics (idempotent merge) rather than raw addition.
- If you need audits: commit to the sparse map (SMT/VC), then prove slot values and transitions.

---

8) Quick decision guide (choose your regime)

- **Symbolic retrieval:** prioritize $\wedge/\vee$, entailment, overlap kernel, indexing.
- **Multi-agent shared memory:** prioritize CRDT counters, sharding, delta sync, convergence proofs.
- **Auditable agent state:** prioritize commitments, proof UX, root timelines, tamper detection.

References (selected)

Gödel encoding, primes, valuations

- Gödel, K. (1931). Über formal unentscheidbare Sätze der Principia Mathematica und verwandter Systeme I. *Monatshefte für Mathematik und Physik*, 38, 173–198.
- Zach, R. (2004). Kurt Gödel's paper "Über formal unentscheidbare Sätze ..." (historical notes and bibliography).

- Fundamental theorem of arithmetic / unique prime factorization (Euclid's lemma; later proofs and history).
- p-adic valuation (definition of v_p as the exponent of p in prime factorization; extension to rationals).
- Avigad, J. (2004). Dedekind's 1871 version of the theory of ideals.

Intersection / overlap kernels for retrieval

- Swain, M. J., & Ballard, D. H. (1991). Color indexing. *International Journal of Computer Vision*, 7, 11–32.
- Barla, A., Odone, F., & Verri, A. (2003). Histogram intersection kernel for image classification. *Proceedings of ICIP*.
- Maji, S., Berg, A. C., & Malik, J. (2008). Classification using intersection kernel SVMs is efficient. *Proceedings of CVPR*.
- Vedaldi, A., & Zisserman, A. (2011/2012). Efficient additive kernels via explicit feature maps. (Intersection/Chi-square/Hellinger kernel maps.)
- Grauman, K., & Darrell, T. (2005). The pyramid match kernel. *Proceedings of ICCV*.

CRDTs and convergent replication

- Shapiro, M., Preguiça, N., Baquero, C., & Zawirski, M. (2011). A comprehensive study of convergent and commutative replicated data types. INRIA Research Report.
- Preguiça, N., Baquero, C., & Shapiro, M. (2018). Conflict-free replicated data types (CRDTs). arXiv: 1805.06358.

Commitments, Merkle trees, sparse Merkle trees

- Merkle, R. C. (1987). A digital signature based on a conventional encryption function. In *CRYPTO*.
- Laurie, B., Langley, A., & Kasper, E. (2013). Certificate transparency. *RFC 6962*.
- Östersjö, R. (2016). Sparse Merkle trees: definitions and space-time trade-offs. (Dissertation/thesis).
- Dahlberg, R., Pulls, T., et al. (2016). Efficient sparse Merkle trees: caching strategies and secure (non-)membership proofs.
- Gao, Z., Hu, Y., & Wu, Q. (2021). Jellyfish Merkle tree. Diem Developers.

Vector commitments, accumulators, SNARKs

- Catalano, D., & Fiore, D. (2013). Vector commitments and their applications.
- Tas, E. N., & Boneh, D. (2023). Vector commitments with efficient updates.
- Benaloh, J., & de Mare, M. (1990s). One-way accumulators: a decentralized alternative to digital signatures.
- Camenisch, J., & Lysyanskaya, A. (2002). Dynamic accumulators and application to efficient revocation of anonymous credentials.
- Groth, J. (2016). On the size of pairing-based non-interactive arguments.
- Loporchio, M., et al. (2023). A survey of set accumulators for blockchain systems.